STREAMPAY

Application Specification & Full System Prompt for AI Agent
Next.js 15 + Stellar SDK Frontend + SDK Package
Drips Wave 11 — Maintainer Application Package

Document Role: This is Document 2 of 2 for StreamPay. Assumes the contract from Document 1 is already built and deployed to Testnet (or a local network). An AI coding agent must implement the entire monorepo exactly as specified. No questions outside this document.

1. Application Goals

A clean, modern web app that lets a user (connected via Freighter or any WalletConnect-compatible Stellar wallet) create a stream, monitor amount owed in real time, pause/resume/cancel, and trigger execute_payment. The app must also expose a minimal TypeScript SDK that any other dApp can import to interact with the StreamPay contract. Indexer is optional for Wave 11 but a simple event listener or polling is required so the UI stays fresh.

2. Exact Monorepo Structure

```
streampay-app/                    # root of the app repo
├── package.json                  # workspaces
├── turbo.json or pnpm-workspace  # optional but preferred
├── packages/
│   └── sdk/                      # @streampay/sdk
│       ├── package.json
│       ├── tsconfig.json
│       ├── src/
│       │   ├── index.ts
│       │   ├── client.ts         # typed wrappers around contract
│       │   ├── types.ts
│       │   └── constants.ts
│       └── README.md
├── apps/
│   └── web/                      # Next.js 15 App Router
│       ├── package.json
│       ├── next.config.ts
│       ├── tailwind.config.ts
│       ├── app/
│       │   ├── layout.tsx
│       │   ├── page.tsx          # dashboard / landing
│       │   ├── create/page.tsx
│       │   ├── stream/[id]/page.tsx
│       │   └── api/              # optional route handlers
│       ├── components/
│       │   ├── WalletConnect.tsx
│       │   ├── CreateStreamForm.tsx
│       │   ├── StreamCard.tsx
│       │   ├── StreamControls.tsx
│       │   └── AmountOwed.tsx
│       ├── lib/
│       │   ├── stellar.ts        # server & client helpers
│       │   └── hooks.ts
│       └── public/
└── README.md                     # how to run, env vars, contract IDs
```

3. Locked Tech Stack

- Next.js 15 (App Router) + TypeScript strict

- Tailwind CSS + shadcn/ui (or plain Tailwind if preferred for speed)
- @stellar/stellar-sdk ^13 or ^14 (latest stable that supports Soroban) — check npm at build time
- @stellar/freighter-api for wallet
- Zustand or React Query for client state
- pnpm or npm workspaces
- No backend database required for MVP; all state from chain + localStorage for UX preferences

4. Environment Variables

Variable	Description	Example
NEXT_PUBLIC_RPC_URL	Soroban RPC endpoint	https://soroban-testnet.stellar.org
NEXT_PUBLIC_NETWORK_PASSPHRASE	Network passphrase	Test SDF Network ; September 2015
NEXT_PUBLIC_CONTRACT_ID	Deployed StreamPay contract ID	C...
NEXT_PUBLIC_USDC_ADDRESS	SEP-41 USDC (or mock) token address	C...
NEXT_PUBLIC_HORIZON_URL	Horizon for account data if needed	https://horizon-testnet.stellar.org

5. SDK Package Specification (@streampay/sdk)

Pure TypeScript package. No React. Exports a StreamPayClient class that takes a Contract ID, RPC URL, and network passphrase. Methods mirror the contract: createStream, pause, resume, updateRate, cancel, executePayment, getStream, amountOwed. Each method returns a prepared TransactionBuilder or a signed XDR string depending on whether a Keypair/wallet is supplied. Also export raw types matching the Stream struct.

```
// packages/sdk/src/client.ts (conceptual)
export class StreamPayClient {
  constructor(opts: {
    contractId: string;
    rpcUrl: string;
    networkPassphrase: string;
  });

  async createStream(params: {
    payer: string;
    payee: string;
    ratePerSecond: bigint;
    source: string;           // account that will sign
  }): Promise<string>;        // XDR or hash

  async amountOwed(streamId: bigint): Promise<bigint>;
  async getStream(streamId: bigint): Promise<Stream>;
  // ... all other entry points
}
```

6. Required UI Flows (must all work end-to-end)

- Connect wallet (Freighter). Show public key and network.
- Create Stream form: payee address, rate (human-readable, convert to base units), optional start note. Submit → sign → submit → show new stream_id and link.
- Stream detail page /stream/[id]: live amount_owed (poll every 5–10 s or on ledger), total paid, status (active/paused/cancelled), controls (Pause/Resume/Cancel/Execute Payment) that appear only for the correct party.

• Dashboard: list of streams where the connected address is payer or payee (query by scanning events or maintaining a simple local index; for MVP a “my streams” list stored in localStorage + manual add by ID is acceptable if full indexer is too heavy).

• Clear error messages for every ContractError.

• Responsive, clean, dark or light theme — professional enough for a grant review demo.

7. Full App System Prompt (Phase 7 — Copy-Paste for Agent)

You are a senior full-stack TypeScript engineer specializing in Stellar Soroban frontends. Your job is to produce a complete, production-quality monorepo for StreamPay that matches this specification. Work in the exact folder structure above. Use conventional commits and push after every logical unit. Never git add .

Numbered Build Sequence:

1. Scaffold monorepo with pnpm workspaces (or npm). Create packages/sdk and apps/web.
2. In packages/sdk: set up TypeScript, export types matching Stream and ContractError, implement StreamPayClient with all methods using @stellar/stellar-sdk Contract and Server (or Rpc).
3. Write unit tests for the SDK using a mock RPC if possible; at minimum type-check cleanly.
4. In apps/web: create Next.js 15 app with App Router, Tailwind, TypeScript strict.
5. Implement wallet connection component using Freighter API. Persist connection state.
6. Implement CreateStreamForm that calls the SDK, requests signature, submits transaction, and redirects to /stream/[id].
7. Implement Stream detail page that polls amountOwed and getStream, shows controls based on connected address vs payer/payee.
8. Implement dashboard that lists known streams (localStorage + ability to paste a stream ID).
9. Add loading, error, and success states for every transaction. Map ContractError codes to human messages.
10. Write a top-level README with: project description, env var table, how to run locally, how to point at a different contract ID, link to the contract repo, and a short demo script (create → wait → execute → pause).
11. Ensure the app builds (next build) with zero TypeScript errors and works against Testnet.
12. Optional but recommended: a simple /api/health route and a Vercel-ready configuration.

Hard Constraints:

- No any types. Strict TypeScript.
- No secrets in client bundle. All sensitive keys stay server-side if ever needed (MVP needs none).
- Wallet interactions must work with Freighter on Testnet.
- All amounts displayed in human units (divide by 10^7 for USDC) but sent as integer base units.
- Accessibility: buttons have labels, forms have proper labels.
- One logical feature = one commit.

8. Demo Script (for reviewers and for you)

1. Open the deployed app → Connect Freighter (Testnet).
2. Create a stream to a second test account at 1000000 base units per second (0.1 USDC/s).
3. Wait 20–30 seconds (or advance ledgers if local).
4. On the stream page click “Execute Payment” → see tokens move.
5. Pause the stream → amount owed stops increasing.
6. Resume → starts accruing again.
7. Cancel from payee side → stream ends.

Record a 60–90 second Loom or short video of this flow; place the link in the README.

9. Acceptance Criteria for Wave Submission

- Contract repo public, builds, tests pass, README has Testnet contract ID and initialize instructions.
- App repo public, builds, README has env example and demo video link.
- Live Testnet deployment of both contract and app (Vercel or similar).
- At least one successful create → execute cycle visible on-chain.
- Clear issues/ labels ready for future contributors (good first issue, bug, enhancement).

END OF DOCUMENT 2 — APPLICATION SPECIFICATION

Together with Document 1 this is a complete, agent-executable package for StreamPay.